

TE608 - Conception de Systèmes Numériques en VHDL2

Lien Github Projet : <https://github.com/Sowker/VHDL2.git>

Lien Github Projet : https://github.com/Sowker/VHDL2.git	1
Introduction :	3
1ère partie :	4
1. Concevoir en VHDL un cœur de microcontrôleur.....	4
1. L'Unité Arithmétique et Logique : ALU/ALU.vhd.....	5
2. Les Mémoires Internes : Buffers/ALUBuffers.vhd.....	6
3. Le Routage des Données : Buffers/ALUSELROUTE.vhd.....	6
4. Le Multiplexeur de Sortie : Memory/MEM_CACHE.vhd.....	7
5. La Mémoire d'Instructions (Le Contrôleur) : Memory/INSTRMemory.vhd.....	8
6. L'Intégration Centrale : Top_Level.vhd.....	9
7. L'Interface Physique : ARTY_Wrapper.vhd.....	9
2. Synthétiser puis tester le microcontrôleur sur la carte de développement ARTY (intégrant un FPGA Artix-35T de Xilinx) en utilisant l'IDE Vivado de Xilinx.....	10
2ème partie :	11
Diviseur de fréquence.....	11
LFSR.....	12
Minuteur	13
LogiGame.....	14
Input handler.....	14
Contrôler.....	15
3ème partie :	18
1.Utiliser le cœur de microcontrôleur pour programmer le LFSR du jeu interactif(remplacement d'une partie du code du jeu).....	18
Conclusion	20

Introduction :

Après une prise en main des différents outils liés à la programmation sur carte FPGA lors de la matière VHDL1, nous passons à VHDL2 afin de mettre en application notre compréhension de la programmation sur carte programmable et de ses contraintes. Pour cela, nous avons pu créer un microcontrôleur de zéro et un jeu fonctionnant sur FPGA. Ce jeu sera un jeu de réflexe où l'utilisateur devra appuyer sur un bouton dans un temps imparti en fonction de la couleur de la LED allumée.

Ce projet se déroule en 3 étapes:

La première étant la création du microcontrôleur. Un composant complexe capable de réaliser plusieurs opérations grâce à son ALU et ses buffers, mais aussi de garder en mémoire ses résultats grâce à sa mémoire cache de 128 bits. Tous ses composants sont reliés par une table d'interconnexion et permettent le bon fonctionnement du microcontrôleur.

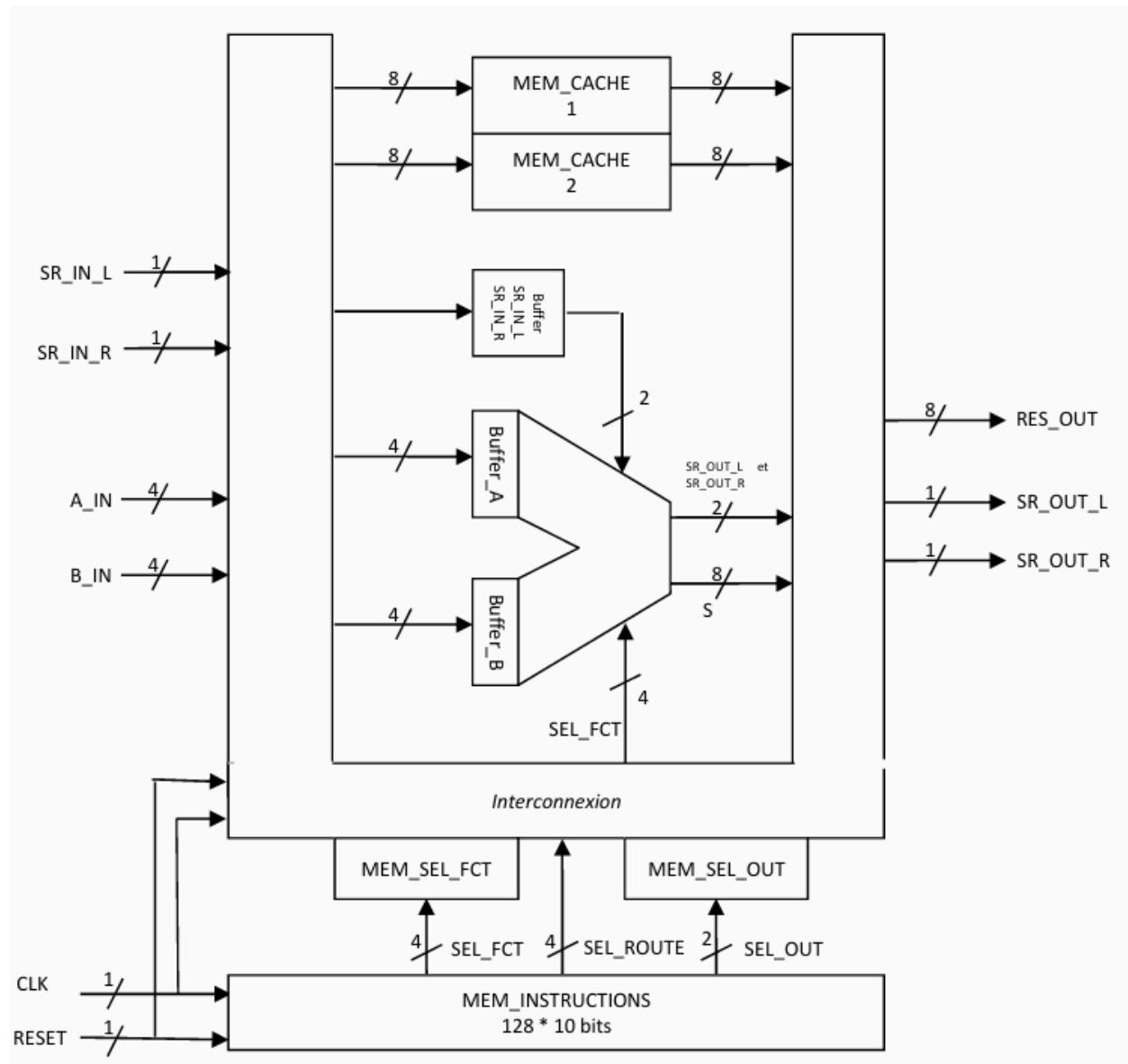
La deuxième partie se focalise sur la création du jeu. Pour ce faire, nous devons créer un composant capable de créer une séquence pseudo aléatoire, le LFSR qui va pouvoir allumer les LED de manière pseudo aléatoire. On va ensuite créer les différents composants permettant le bon déroulement du jeu comme le Minuteur, LogiGame ou bien encore l'Input Controller qui seront des composants pour le jeu. Celui-ci prendra la forme d'un automate dans le fichier Contrôler.

Enfin, la troisième et dernière partie permet la mise en commun des deux parties précédentes. Ainsi, à travers celle-ci, nous allons générer la séquence pseudo aléatoire par le micro-contrôleur, permettant de créer un jeu complet.

1ère partie :

1. Concevoir en VHDL un cœur de microcontrôleur

Le plus gros challenge pour nous dans la conception du cœur de microcontrôleur simple a vraiment été la compréhension de l'architecture cible et l'accumulation des différentes informations par rapport à la mémoire et aux instructions.



Pour mener à bien cette conception, nous avons découpé l'implémentation VHDL en plusieurs sous-modules, chacun ayant un rôle précis dans le traitement, le stockage ou le routage des données. Voici la description détaillée de l'arborescence de notre microcontrôleur :

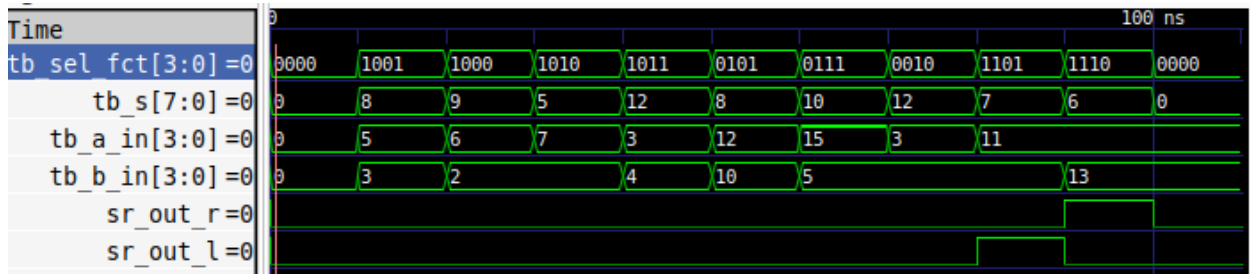
1. L'Unité Arithmétique et Logique : ALU/ALU.vhd

Le fichier ALU/ALU.vhd contient l'entité ALUCore, qui est le cœur de calcul du microcontrôleur.

- **Rôle** : Il s'agit d'un bloc purement combinatoire (sans signal d'horloge) chargé d'effectuer les opérations arithmétiques (addition, soustraction, multiplication) et logiques (AND, OR, XOR, décalages) selon la valeur de Sel_FCT et des deux entrées A_IN et B_IN comme indiqué dans ce tableau :

SEL_FCT[3]	SEL_FCT[2]	SEL_FCT[1]	SEL_FCT[0]	Significations
0	0	0	0	nop (no operation) S = 0 SR_OUT_L = 0 et SR_OUT_R = 0
0	0	0	1	S = A SR_OUT_L = 0 et SR_OUT_R = 0
0	0	1	0	S = not A SR_OUT_L = 0 et SR_OUT_R = 0
0	0	1	1	S = B SR_OUT_L = 0 et SR_OUT_R = 0
0	1	0	0	S = not B SR_OUT_L = 0 et SR_OUT_R = 0
0	1	0	1	S = A and B SR_OUT_L = 0 et SR_OUT_R = 0
0	1	1	0	S = A or B SR_OUT_L = 0 et SR_OUT_R = 0
0	1	1	1	S = A xor B SR_OUT_L = 0 et SR_OUT_R = 0
1	0	0	0	S = A + B addition binaire avec SR_IN_R comme retenue d'entrée
1	0	0	1	S = A + B addition binaire sans retenue d'entrée
1	0	1	0	S = A - B soustraction binaire
1	0	1	1	S = A * B multiplication binaire
1	1	0	0	S = Déc. droite A sur 4 bits (avec SR_IN_L) SR_IN_L pour le bit entrant et SR_OUT_R pour le bit sortant
1	1	0	1	S = Déc. gauche A sur 4 bits (avec SR_IN_R) SR_IN_R pour le bit entrant et SR_OUT_L pour le bit sortant
1	1	1	0	S = Déc. droite B sur 4 bits (avec SR_IN_L) SR_IN_L pour le bit entrant et SR_OUT_R pour le bit sortant
1	1	1	1	S = Déc. gauche B sur 4 bits (avec SR_IN_R) SR_IN_R pour le bit entrant et SR_OUT_L pour le bit sortant

- **Testbench (tb_ALUCore.vhd):**



Pour prouver le bon fonctionnement de ALUCore, nous avons appliqué une valeur différente de Sel_FCT toutes les 10 ns. L'analyse des signaux de sortie confirme un parfait fonctionnement de l'ALU :

- **0 à 10 ns - NOP** : Sel_FCT à "0000". La sortie S est forcée à "00000000" et les retenues SR_OUT_L/R restent à '0'.
- **10 à 20 ns - Addition standard (A + B)** : Sel_FCT à "1001". Avec A = 5 et B = 3, la sortie S bascule instantanément à 8 ("00001000").
- **20 à 30 ns - Addition avec retenue** : Sel_FCT à "1000". Avec A = 6, B = 2 et la retenue d'entrée SR_IN_R = '1', la sortie S donne bien 9 ("00001001").
- **30 à 40 ns - Soustraction (A - B)** : Sel_FCT à "1010". Avec A = 7 et B = 2, la sortie S affiche correctement la différence de 5 ("00000101").
- **40 à 50 ns - Multiplication (A × B)** : Sel_FCT à "1011". Étape clé de notre premier automate, le produit de A = 3 par B = 4 est validé à 12 ("00001100").

- **50 à 60 ns - ET Logique (AND) :** Sel_FCT à "0101". Pour A = "1100" et B = "1010", l'ALU réalise un ET bit à bit exact et renvoie 8 ("00001000").
- **60 à 70 ns - OU Exclusif (XOR) :** Sel_FCT à "0111". Utile pour notre deuxième automate, l'opération entre "1111" et "0101" donne immédiatement 10 ("00001010").
- **70 à 80 ns - Inversion (NOT A) :** Sel_FCT à "0010". Avec A = "0011", la sortie S renvoie bien le complément de A qui est 12 ("00001100").
- **80 à 90 ns - Décalage Gauche :** Sel_FCT à "1101". Le vecteur A = "1011" est décalé à gauche avec l'injection de SR_IN_R = '1'. Le bit de poids fort ('1') est éjecté avec succès sur SR_OUT_L. S affiche "00000111".
- **90 à 100 ns - Décalage Droite :** Sel_FCT à "1110". Le vecteur B = "1101" est décalé à droite avec SR_IN_L = '0'. Le bit de poids faible ('1') est éjecté sur SR_OUT_R. S se stabilise à "00000110".

2. Les Mémoires Internes : Buffers/ALUBuffers.vhd

L'entité ALUBuffer sert de zone de stockage temporaire pour le microcontrôleur.

- **Rôle :** Elle sauvegarde les résultats intermédiaires lorsque l'automate (INSTRMemoire.vhd) doit réaliser des calculs complexes qui prennent plusieurs étapes (comme la multiplication).
- **Fonctionnement :** Ce bloc regroupe plusieurs cases mémoires : deux de 4 bits pour les entrées (Buffer_A et Buffer_B) et deux de 8 bits pour les résultats (Mem_1 et Mem_2). Pour des raisons de synchronisation, une donnée n'est enregistrée que si deux conditions sont réunies : le rythme de l'horloge (CLK) donne le top départ, et le signal d'autorisation correspondant (CE_Buf) est activé.

3. Le Routage des Données : Buffers/ALUSELROUTE.vhd

- **Rôle :** Elle détermine quelles données doivent être écrites dans quels registres du bloc ALUBuffer pour différentes valeurs de SEL_ROUTE comme indiqué dans ce tableau :

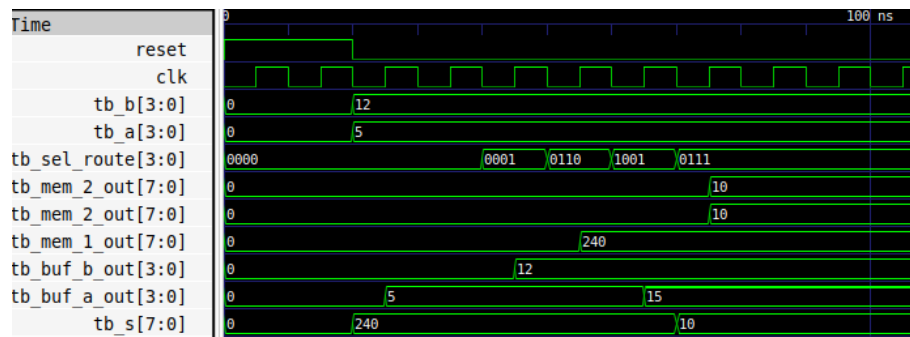
SEL_ROUTE[3]	SEL_ROUTE[2]	SEL_ROUTE[1]	SEL_ROUTE[0]	Significations
0	0	0	0	Stockage de l'entrée A_IN dans Buffer_A
0	0	0	1	Stockage de l'entrée B_IN dans Buffer_B
0	0	1	0	Stockage de S dans Buffer_A (4 bits de poids faibles)
0	0	1	1	Stockage de S dans Buffer_A (4 bits de poids forts)
0	1	0	0	Stockage de S dans Buffer_B (4 bits de poids faibles)
0	1	0	1	Stockage de S dans Buffer_B (4 bits de poids forts)
0	1	1	0	Stockage de S dans MEM_CACHE_1
0	1	1	1	Stockage de S dans MEM_CACHE_2
1	0	0	0	Stockage de MEM_CACHE_1 dans Buffer_A (4 bits de poids faibles)
1	0	0	1	Stockage de MEM_CACHE_1 dans Buffer_A (4 bits de poids forts)
1	0	1	0	Stockage de MEM_CACHE_1 dans Buffer_B (4 bits de poids faibles)
1	0	1	1	Stockage de MEM_CACHE_1 dans Buffer_B (4 bits de poids forts)
1	1	0	0	Stockage de MEM_CACHE_2 dans Buffer_A (4 bits de poids faibles)
1	1	0	1	Stockage de MEM_CACHE_2 dans Buffer_A (4 bits de poids forts)
1	1	1	0	Stockage de MEM_CACHE_2 dans Buffer_B (4 bits de poids faibles)
1	1	1	1	Stockage de MEM_CACHE_2 dans Buffer_B (4 bits de poids forts)

Testbench pour tester ALUSELROUTE.vhd et ALUBuffer.vhd

(tb_ALUMemoryRoute.vhd) :

Afin de vérifier la bonne communication entre ALUSELROUTE et ALUBuffer, nous avons créé un testbench liant les deux. Contrairement à l'ALU, ce sous-système est séquentiel.

L'analyse de la simulation valide le bon fonctionnement des deux modules :



- **Reset** : L'activation du signal RESET force initialement l'ensemble des mémoires (Buffer_A, Buffer_B, Mem_1, Mem_2) à zéro.
- **Routing de A et B** : En appliquant SEL_ROUTE = "0000", le routeur active CE_Buf_A et présente la valeur de l'entrée A ("0101"). Au front d'horloge suivant, Buffer_A mémorise bien cette valeur. La même logique est validée avec SEL_ROUTE = "0001" qui vient charger l'entrée B ("1100") dans Buffer_B.
- **Stockage en Cache** : Avec SEL_ROUTE = "0110", la sortie de l'ALU s (fixée à "11110000") est redirigée vers le cache principal. Au cycle suivant, Mem_1 met à jour sa valeur en conséquence.
- **Rebouclage des mémoires** : Test critique pour nos automates, nous appliquons SEL_ROUTE = "1001". Le routeur extrait correctement les 4 bits de poids forts de Mem_1 ("1111") et les redirige vers Buffer_A.

4. Le Multiplexeur de Sortie : Memory/MEM_CACHE.vhd

Ce fichier abrite l'entité MEM_SEL_OUT_MUX.

- **Rôle** : Il a pour fonction de définir ce qui sera présenté sur la sortie finale RES_OUT du microcontrôleur en fonction de SEL_OUT comme indiqué dans ce tableau :

SEL_OUT[1]	SEL_OUT[0]	Significations
0	0	Aucune sortie : RES_OUT = 0
0	1	RES_OUT = MEM_CACHE_1
1	0	RES_OUT = MEM_CACHE_2
1	1	RES_OUT = S

5. La Mémoire d'Instructions (Le Contrôleur) : `Memory/INSTRMemory.vhd`

L'entité `INSTRMemory` constitue le "cerveau" ou l'automate de pilotage de notre architecture.

- **Rôle** : Elle stocke le programme (les séquences d'instructions) et gère le déroulement des calculs étape par étape.
- **Fonctionnement** : Elle intègre une mémoire contenant jusqu'à 128 instructions de 10 bits. Chaque ligne de 10 bits est divisée pour générer les signaux de contrôle du système : `SEL_ROUTE` (bits 9 à 6), `SEL_FCT` (bits 5 à 2) et `SEL_OUT` (bits 1 et 0). L'automate gère un pointeur d'instruction (`pc`) qui s'incrémente ou saute à des adresses spécifiques (1, 5 ou 13) lors de l'appui sur les boutons `BTN_1`, `BTN_2` ou `BTN_3` pour lancer les 3 automates demandés (Multiplication, XNOR...). Il intègre également des états d'arrêt pour bloquer l'exécution à la fin d'un calcul.

Illustration avec l'Automate 1 (Multiplication) : Pour réaliser une multiplication, l'automate orchestre le système sur quatre cycles d'horloge consécutifs en émettant des ordres spécifiques à chaque étape :

- **Étape 1 (Acquisition du premier opérande) — Instruction "0000 0000 00"** : Le contrôleur émet une instruction de routage (`SEL_ROUTE` = "0000") qui ordonne de lire la première valeur en entrée et de la verrouiller dans le `Buffer_A`. L'ALU reste au repos (`SEL_FCT` = "0000") et la sortie est inactive (`SEL_OUT` = "00").
- **Étape 2 (Acquisition du second opérande) — Instruction "0001 0000 00"** : Au cycle suivant, une nouvelle commande de routage (`SEL_ROUTE` = "0001") est envoyée pour acquérir la seconde valeur et la stocker dans le `Buffer_B`. L'ALU est toujours au repos (`SEL_FCT` = "0000"). Le système dispose désormais de ses deux opérandes en mémoire interne.
- **Étape 3 (Calcul et sauvegarde) — Instruction "0110 1011 00"** : C'est la phase de traitement. Le contrôleur envoie simultanément deux ordres croisés : il configure l'ALU pour qu'elle multiplie le contenu des deux buffers (`SEL_FCT` = "1011"), et il ordonne au routeur de capturer le résultat sortant de l'ALU pour l'inscrire directement dans une mémoire de sauvegarde `MEM_CACHE_1` (`SEL_ROUTE` = "0110").
- **Étape 4 (Affichage final) — Instruction "0000 0000 01"** : Le calcul est achevé. Les modules de calcul et de routage repassent au repos (`SEL_ROUTE` = "0000", `SEL_FCT` = "0000"). L'automate utilise le signal de sortie (`SEL_OUT` = "01") pour paramétrer le multiplexeur final. Ce dernier va lire la valeur stockée dans le cache et la présenter sur les sorties principales du processeur (`RES_OUT`).

Difficulté : L'absence de point d'arrêt (Fall-Through) Lors des tests sur carte, notre processeur affichait des résultats tout seul ou n'arrêtait pas de calculer. On a compris que notre pointeur d'instruction (`pc`) tournait en roue libre à 100 MHz ! Pour corriger ce bug majeur, on a dû créer des états "HALT" : on a forcé le compteur à boucler sur lui-même à la fin de chaque

programme (ex : `elsif pc = 4 then pc <= 4;`) pour l'obliger à s'arrêter et à attendre l'utilisateur.

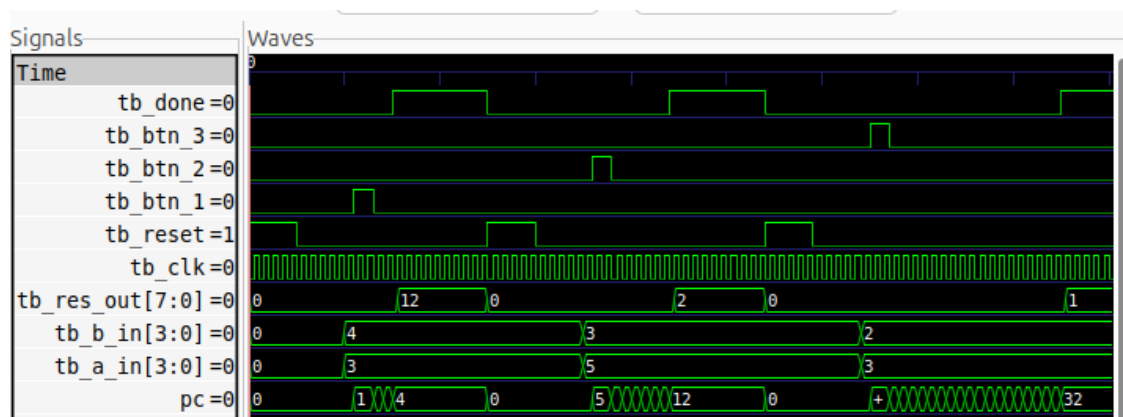
6. L'Intégration Centrale : `Top_Level.vhd`

- **Rôle** : Le `Top_Level` est le composant structurel qui relie physiquement toutes les entités mentionnées ci-dessus.
- **Fonctionnement** : C'est ici que s'effectue le câblage entre la mémoire d'instructions, le cœur de l'ALU, les registres et le routeur via des signaux internes. Il expose les entrées/sorties génériques du microcontrôleur (`A_IN`, `B_IN`, `RES_OUT`, `DONE`, etc.). Nous avons aussi créé la sortie `DONE` passant à '1' lorsque le code `SEL_OUT` indique qu'un résultat est prêt à être affiché.

7. L'Interface Physique : `ARTY_Wrapper.vhd`

- **Rôle** : Adapter et lier les ports de notre `Top_Level` à la carte FPGA Digilent ARTY A7.
- **Fonctionnement** : Ce fichier mappe les interrupteurs de la carte aux entrées `A_IN` et `B_IN` (dans notre cas `A_IN = B_IN`). Les boutons poussoirs sont reliés au reset global et aux déclenchements des 3 automates. Enfin, il répartit l'affichage du résultat 8 bits (`sig_res_out`) à la fois sur les 4 LEDs vertes standard et sur la composante rouge des 4 LEDs RGB, tout en utilisant la couleur bleue pour afficher l'état de complétion (`DONE`) et les retenues (`SR_OUT_L / R`).

Tesbench pour tester `INSTRMemory.vhd`, `Top_Level.vhd` et `ARTY_Wrapper.vhd` (`tb_Top_Level.vhd`) :



Le chronogramme de simulation valide l'exécution correcte de nos trois programmes présents dans `INSTRMemory` :

- **Validation de l'Automate 1 (Bouton 1)** : Nous avons affecté `A=3` et `B=4`, puis simulé une impulsion sur `BTN_1`. Le pointeur d'instruction (`pc`) saute à l'adresse 1 et exécute séquentiellement le programme de multiplication. Après 4 cycles d'horloge, le signal `DONE` passe à '1' et la sortie finale `RES_OUT` affiche la valeur 12 sur 8 bits, validant le traitement multitâche.

- **Validation de l'Automate 2 (Bouton 2) :** Après un Reset, nous plaçons A=5 et B=3, et pressons BTN_2. L'architecture enclenche une suite plus longue d'instructions : calcul de l'addition A+B (8), rebouclage dans les buffers, application d'un XOR avec A (donnant 13), rebouclage, puis application d'un NOT pour réaliser le XNOR final. Le résultat res_out affiché lors de l'apparition du signal DONE est "00000010" (2), ce qui est strictement exact.
- **Validation de l'Automate 3 (Bouton 3) :** Simulant l'opération de logique combinatoire complexe (A_0 AND B_1) OR (A_1 AND B_0) exigée dans le cahier des charges, la pression sur BTN_3 déclenche un programme d'une vingtaine de cycles. Les signaux de routage s'enchaînent de manière autonome sans intervention extérieure jusqu'à la présentation du bon résultat sur RES_OUT.

2. Synthétiser puis tester le microcontrôleur sur la carte de développement ARTY (intégrant un FPGA Artix-35T de Xilinx) en utilisant l'IDE Vivado de Xilinx

Pour terminer cette première partie, nous sommes passés sur Vivado pour tester notre code en conditions réelles sur le FPGA de la carte ARTY. En combinant notre code global (Top_Level et ARTY_Wrapper) avec le fichier de contraintes fourni pour le projet, nous avons lancé la synthèse. Tout s'est bien passé et Vivado a réussi à compiler le projet et à générer le bitstream.

Une fois le programme chargé sur la carte, nous avons fait nos tests physiques avec les boutons et les switches. Résultat : tout marchait nickel ! Le microcontrôleur réagissait exactement de la même manière que sur nos simulations.

Note : Le fichier bitstream ainsi qu'une petite vidéo prise avec nos téléphones pour bien montrer que le système fonctionne en vrai sont inclus dans l'archive ZIP du rendu.

2ème partie :

- Concevoir en VHDL un jeu interactif «simple »
- Synthétiser puis tester le jeu sur la carte de développement ARTY (intégrant un FPGA Artix-35T de Xilinx) en utilisant l'IDE Vivado de Xilinx

Diviseur de fréquence

Le diviseur de fréquence est un composant capable de diviser la fréquence d'origine afin qu'un futur composant puisse fonctionner sur une fréquence plus basse.

Ainsi, c'est un générateur de fréquence qui génère une fréquence tous les X cycles. Pour cela, il compte le nombre de cycles avant de s'actionner. On appelle ce composant le diviseur de fréquence car il permet de réduire la fréquence reçue tel que l'output est égal à la l'input divisé par le nombre de cycle compté par le compteur.

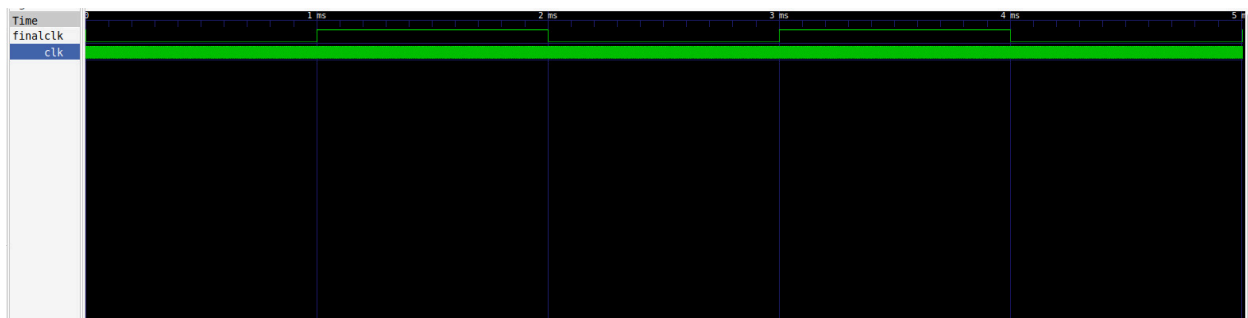
Dans notre cas, nous avons eu besoin de faire passer notre fréquence de 100 MHz à 1 kHz, ce qui a nécessité la division du signal initial par 100 000. Ainsi, nous devons compter 100 000 cycles avant d'envoyer un signal positif en sortie et ainsi de suite.

Il prend en signal d'entrée,

- la clock de 100 MHz

Et ressort

- un signal périodique de 1 kHz



Sur ce graphe, on peut voir 2 signaux, notre clock d'entrée qui fait X cycles en 1 ms après quoi le notre compteur a atteint sa limite et génère un signal positif en sortie et se réinitialise avant de re compter les cycles afin de désactiver le signal de sortie. Ce cycle du compteur est répété à l'infini pour permettre la division de la fréquence.

LFSR

Le registre à décalage à rétroaction linéaire ou LFSR est un composant capable de générer une séquence pseudo aléatoire grâce à une suite de d flip flop et d'un feedback modélisé par un XOR entre le bit 2 et 3. Ce polynôme pseudo aléatoire basé sur 4 bits, tel que $P(x) = x^4 + x^3 + 1$.

Ce composant permettra de déterminer la couleur de la LED qui s'allumera aléatoirement tel que le résultat de $P(x)$ modulo 3 représente une couleur de LED.

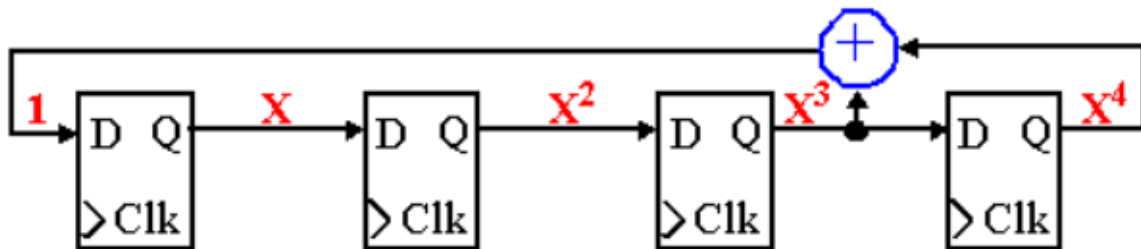
Il prend en signaux d'entrées,

- la clock de 1kHz
- un signal Reset sur un bit qui le réinitialiser à sa valeur de base "1011"
- un signal Enable sur un bit qui l'actionne

Et renvoie,

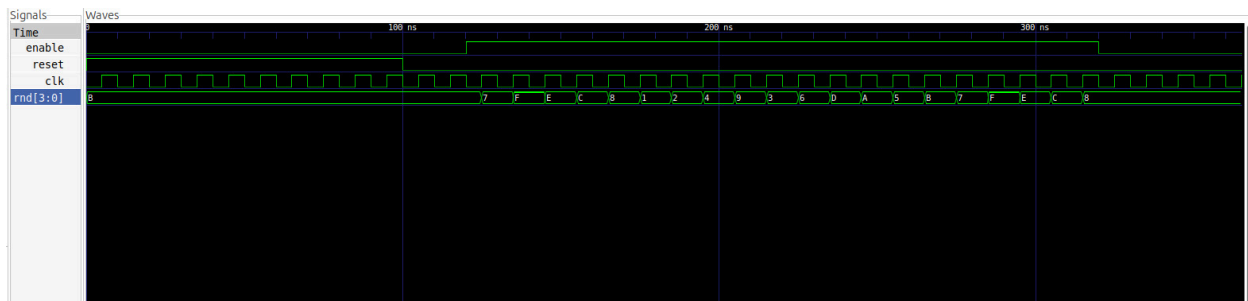
- un signal sur 4 bit représentant le polynôme

Schéma du composant:



À noter, la valeur de base du composant est "1011", il faut donc faire attention lorsque nous utiliserons le composant en vérifiant qu'il génère bien une nouvelle séquence.

Pour le faire fonctionner, nous utilisons une variable "enable" qui permet de l'actionner et qui lui permet de performer ses calculs.



Dans ce chronogramme, nous pouvons voir comment le LFSR fonctionne. Tout d'abord, on peut voir que l'entrée Reset fonctionne très bien et force la valeur du LFSR à B, soit 1011. Lorsqu'il est activé, il performe enfin ses calculs et change la valeur de la sortie du LFSR suivant la formule de $P(1011)=0111$, ce qui est bien 7, puis continue après chaque front montant. Ainsi, après on a $P(0111)=1111$ et ainsi de suite.

Minuteur

Le but du composant minuteur est de compter le nombre de cycles de la clock à 100 MHz en fonction de la difficulté encodée sur les bits 3 et 2 du signal SW et choisie par le joueur tel que le tableau suivant

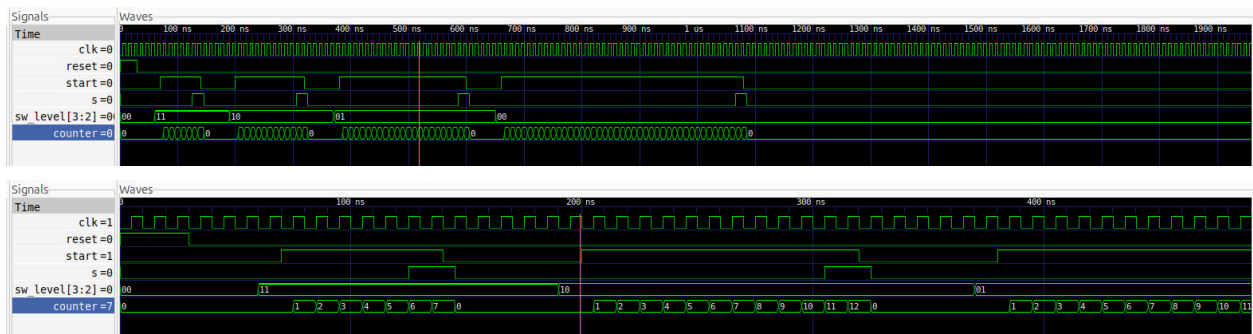
SW[3:2]	Niveau	Nombre de cycles (ex.)	Durée approx. à 100 MHz
00	Très facile	400 000 000	4 secondes
01	Facile	200 000 000	2 secondes
10	Moyen	100 000 000	1 seconde
11	Difficile	50 000 000	0,5 seconde

Il prend en signal d'entrée,

- une clk de 100 MHz
- un signal Reset qui remet à zéro le compteur et le signal timeout
- un signal Start qui lance le compteur
- un signal sw_level qui tient les 4 difficultés du jeu

Et à comme sortie

- timeout passe à '1' lorsque le délai programmé est atteint que j'ai appelé S



Dans ce chronogramme, on peut voir que le minuteur s'active bien lorsque la difficulté est sélectionnée et compte le nombre de cycles. De plus, on peut voir que suivant la difficulté, le signal timeout S s'active à différents nombres de cycles, représentant les différents nombres de difficultés.

LogiGame

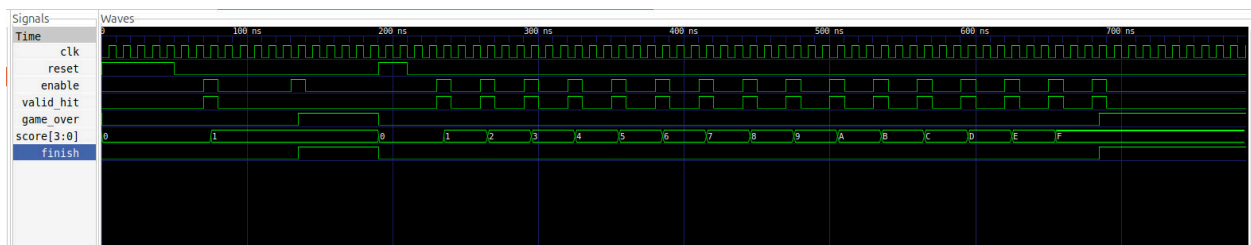
LogiGame est un composant qui permet de calculer le score du joueur en fonction de ses actions. Ce composant implémente l'action du joueur avec un signal d'entrée encodé sur un bit valid_hit. Si le score atteint 15, alors le joueur a fini le jeu et la variable game_over s'active. Cependant, s'il se trompe, la sortie game_over s'active aussi. Ainsi, game_over symbolise la fin du jeu mais ne distingue pas si le joueur a fini le jeu ou a perdu. Pour voir la différence il faut regarder le score affiché sur les LED.

Signal d'entrée,

- Entrée clk : horloge système
- Entrée reset : remise à zéro du score
- Entrée valid_hit : indiquant la réussite (1) ou l'échec (0)

Signal de sortie

- Sortie score : score courant codé sur 4 bits
- Sortie game_over : signal indiquant la fin du jeu



Dans ce chronogramme, nous pouvons voir que le logigame fonctionne parfaitement, et qu'à chaque valid_hit, le score est incrémenté de 1 jusqu'à 15, le score maximal qui fait alors passer le composant en finish (booléen assigné à True quand le jeu est fini). De plus, si lors du jeu il se trompe et qu'il fait un game over, le jeu passe directement en mode finish et arrête le jeu. Ainsi, nous pouvons voir que le composant implémente bien la logique du jeu.

Input handler

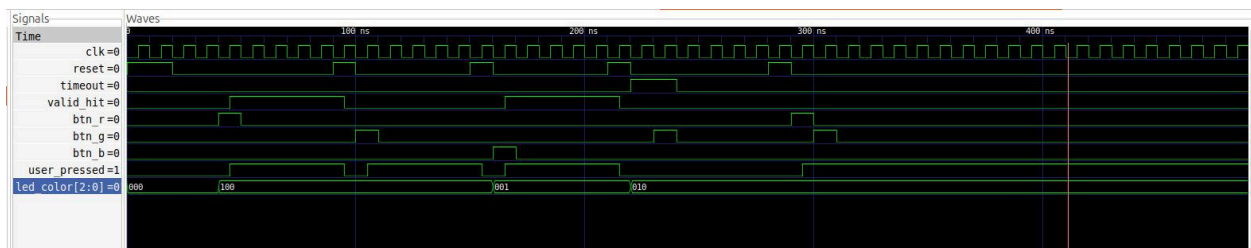
Input handler est un composant qui va s'occuper de vérifier si le joueur appuie sur le bon bouton et au bon moment.

Signal d'entrées

- Entrée clk : horloge système
- Entrée reset : réinitialisation du module
- Entrée timeout : signal de fin de délai
- Entrée led_color : couleur affichée sur LD3 (3 bits, R=100, G=010, B=001)
- Entrées btn_r, btn_g, btn_b : boutons de réponse (BTN1, BTN2, BTN3)

Signal de sortie

- Sortie valid_hit : passe à '1' si la bonne réponse a été donnée dans les temps



Sur ce chronographe, nous pouvons voir l'Input Handler en action, capable de déterminer si l'utilisateur appuie sur le bouton en accord avec la LED. Ainsi, nous pouvons voir que si le joueur appuie sur le bouton en accord avec la LED, l'output `valid_hit` passe à True, renvoyant le un signal signalant que le joueur a appuyé sur le bon bouton (de 60 à 230 ns). Cependant, si le joueur appuie sur le mauvais bouton comme à 140 ns, ou il appuie sur le bouton vert alors que la LED rouge s'allume, alors rien ne se passe et le `valid_hit` n'est pas activé. Après 230 ns, on teste avec le test bench ce qu'il se passe si le joueur appuie trop tard sur le bouton. Parce que `timeout` est déjà à True, alors qu'importe le bouton appuyé par le joueur qu'il soit bon ou pas, le `valid_hit` ne passe pas à True.

Contrôler

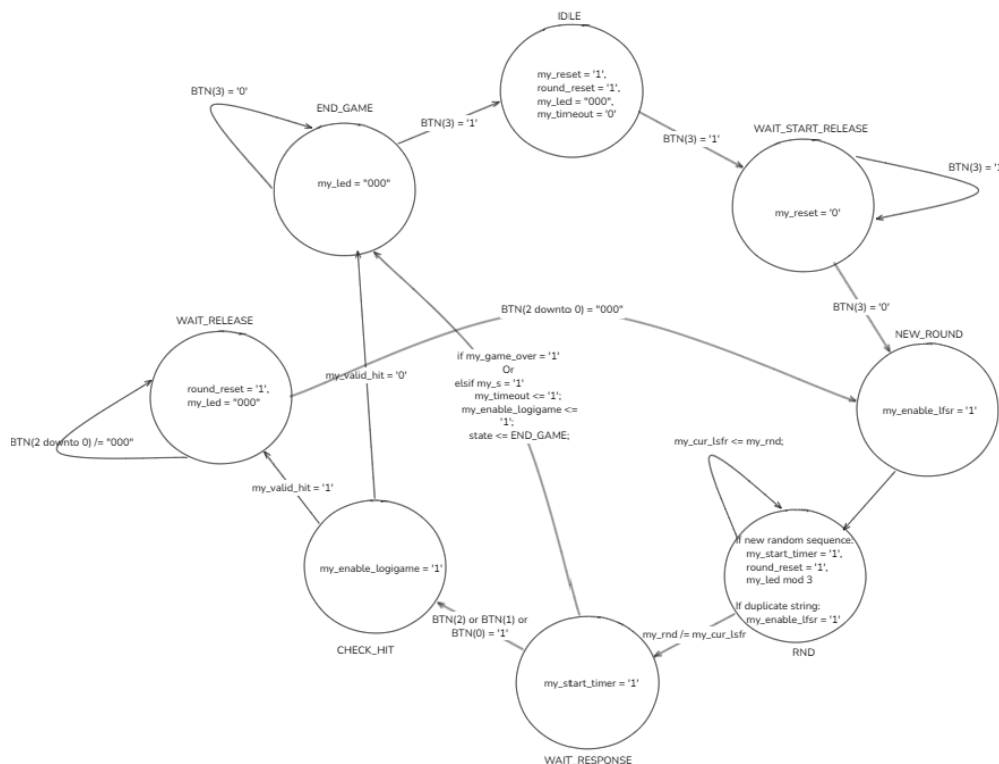
Enfin, le composant Contrôler est le composant qui tient la logique générale du jeu en liant les différents composants entre eux. Il se base sur un automate avec différents états et permet ainsi au joueur de jouer indéfiniment.

Voici comment l'automate fonctionne :

- **Initialisation (IDLE / WAIT_START_RELEASE)** : Le contrôleur maintient un Reset asynchrone sur les modules (`my_reset <= '1'`). Il utilise deux états pour sécuriser le démarrage et s'assurer que le bouton de lancement BTN(3) génère bien un front descendant propre (relâchement physique). J'aurais dû utiliser un debounce comme vous l'avez suggéré mais je pensais que c'était possible sans mais non ! Du coup, je dois utiliser 2 Etat pour vérifier la fin du maintien du bouton.
- **Traitement Aléatoire (RND)** : Le FSM lit la sortie de 4 bits du LFSR (`my_rnd`), utilise une opération mathématique combinatoire (mod 3) pour router ce nombre vers un codage modulo 3 bits afin d'allumer une seule LED (R, G ou B). Le registre `my_cur_lsfr` sert de buffer pour comparer le nouveau nombre à l'ancien, empêchant ainsi matériellement que la même séquence/couleur n'apparaisse deux fois de suite.
- **Acquisition (WAIT_RESPONSE)** : C'est l'état d'attente du jeu, l'automate attend soit que le joueur ait appuyé dans un temps imparti soit que le temps soit écoulé. La FSM écoute trois signaux en parallèle : le bit de timeout (`my_s`) ce qui signifie que le joueur n'a pas appuyé dans le temps imparti, l'état global du jeu (`my_game_over`), et les

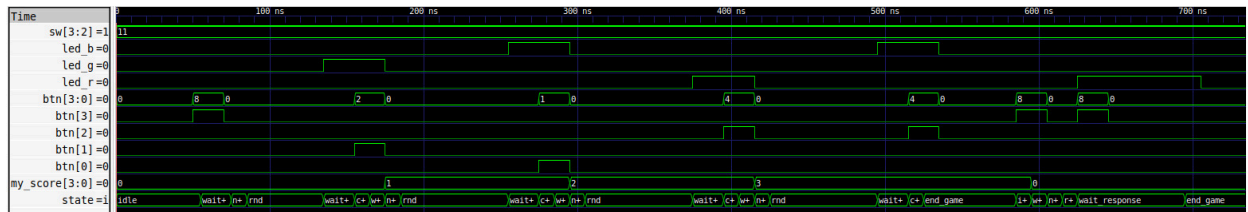
boutons BTN(0), BTN(2) ou BTN(3) signifiant que le joueur à jouer dans le temps impartie.

- **Latence de Propagation (CHECK_HIT) :** C'est un état qui va vérifier l'action du joueur. Lorsqu'un bouton est pressé, la FSM passe dans cet état pour un unique cycle d'horloge. Cela laisse le temps de propagation combinatoire nécessaire au module InputHandler pour évaluer les entrées et stabiliser le bit my_valid_hit avant que la FSM ne prenne une décision au coup d'horloge suivant. Soit en renvoyant vers WAIT_RELEASE car le joueur à appuyé sur le bon bouton my_valid_hit = '1' ou vers END_GAME s'il s'est trompé et à perdu my_valid_hit = '0'.
- **Debouncer (WAIT_RELEASE) :** Cet état comme le debouncer précédent vérifie que le joueur est lâché le bouton avant de relancer le prochain round et ne pas mettre l'ancienne action du joueur dans les actions à évaluer.



Il est important de noter que nous ne savons pas faire de debouncer et ne savons pas comment l'implémenter, donc nous avons préféré implémenter ce composant à travers des états qui vérifient que le joueur n'appuie plus sur le bouton.

De plus, de par notre implémentation du LSFR, la séquence aléatoire n'est pas aléatoire. Ainsi, il faudrait permettre de prendre en compte le temps de réaction de l'utilisateur pour générer la séquence pseudo aléatoire, mais cela n'est pas assez explicite pour nous.



Sur le chronogramme, nous pouvons voir une simulation d'une tentative d'une partie du jeu qui se finit avec un score de 3. Nous pouvons voir que l'automate fonctionne très bien puisqu'il génère les LED de manière aléatoire. De plus, il gère très bien l'incrémentation du score et les nouveaux rounds.

A note que pouvoir simuler ce testbench, j'ai du modifier la valeur du diviseur de fréquence et du minuteurs afin de compter moins de cycle

3ème partie :

1.Utiliser le cœur de microcontrôleur pour programmer le LFSR du jeu interactif(remplacement d'une partie du code du jeu)

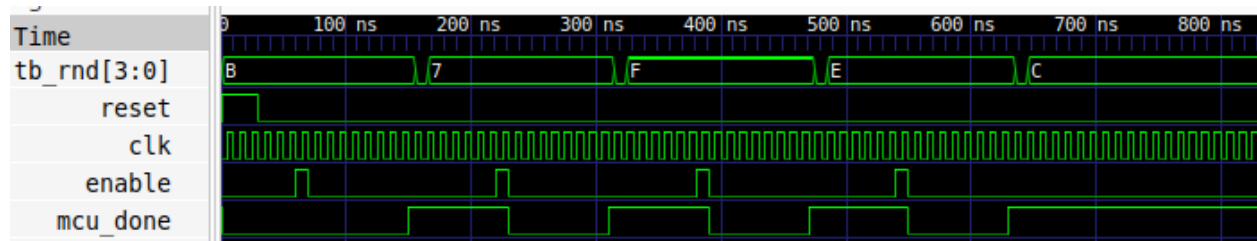
Pour cette 3ème et dernière partie, l'objectif était l'aboutissement final de notre projet : prouver que le processeur que nous avons créé de toutes pièces dans la Partie 1 est capable de remplacer un composant matériel dédié du jeu (le LFSR de la Partie 2). Au lieu d'avoir un circuit électronique fixe qui calcule la suite pseudo-aléatoire, c'est notre microcontrôleur qui va exécuter un programme codé dans sa mémoire d'instruction.

Sur le papier, le calcul du LFSR a l'air simple : il faut faire un XOR entre le bit 3 et le bit 2, puis décaler le tout vers la gauche en insérant le résultat du XOR à droite. Sauf que notre ALU ne sait pas faire un XOR entre deux bits précis d'un même mot ! Elle fait des XOR entre deux vecteurs de 4 bits. Il a donc fallu ruser et créer un algorithme sur 9 cycles d'horloge pour "isoler" ce bit :

1. On charge la valeur actuelle du LFSR dans le `Buffer_A`.
2. On décale cette valeur d'un cran vers la droite et on la stocke dans le `Buffer_B`.
3. On demande à l'ALU de faire `Buffer_A XOR Buffer_B`. À cet instant précis, le résultat de `bit 3 XOR bit 2` est caché au milieu du résultat ! On stocke ça dans le `Buffer_B`.
4. On décale le `Buffer_B` vers la droite 3 fois de suite. Pourquoi ? Pour forcer notre fameux bit caché à "tomber" en dehors du buffer et à être récupéré par la retenue sortante de l'ALU (`SR_OUT_R`).
5. L'astuce finale : on ordonne un décalage à gauche du `Buffer_A`. L'ALU va naturellement "aspirer" la retenue (qui contient notre XOR) et l'insérer dans le bit de poids faible. Le tour est joué, on a notre nouvelle valeur LFSR ! On la sort sur `RES_OUT`.

Pour intégrer ça sans casser tout le code du jeu, nous avons créé `LFSR_MCU.vhd`. Ce fichier se comporte comme l'ancien LFSR (mêmes entrées/sorties), mais à l'intérieur, il contient notre `Top_Level`. Quand le jeu envoie le signal `ENABLE`, le Wrapper appuie virtuellement sur le bouton `BTN_1` de notre processeur. Le processeur exécute son programme de 9 lignes, active `DONE`, et `LFSR_MCU` renvoie le nouveau nombre aléatoire au jeu.

Testbench du `LFSR_MCU` (`tb_LFSR_MCU.vhd`) :



Pour prouver que notre microcontrôleur effectue correctement le travail, nous avons réalisé un nouveau testbench pour isoler `LFSR_MCU`. L'objectif était de vérifier que notre algorithme en 9 cycles produisait rigoureusement la même suite mathématique que le composant matériel d'origine, à savoir la suite basée sur le polynôme $P(x) = x^4 + x^3 + 1$.

L'analyse du chronogramme de simulation valide notre approche :

- **Initialisation** : Lors du `RESET`, la valeur par défaut est bien fixée à "1011".
- **Génération 1** : On envoie une impulsion sur `ENABLE`. Contrairement à la Partie 2 où le résultat changeait instantanément au front d'horloge suivant, on observe ici une latence d'environ 90 ns (9 cycles d'horloge). C'est le temps exact qu'il faut à notre microcontrôleur pour faire ses décalages et ses XOR. À la fin du calcul, le signal passe bien à "0111".
- **Générations suivantes** : En relançant `ENABLE`, on obtient successivement "1111", puis "1110", puis "1100".

Séquence aléatoire (B -> 7 -> F -> E -> C)

La simulation prouve que notre détournement matériel (isoler un bit dans la retenue pour le réinjecter via un décalage) fonctionne parfaitement. Notre cœur de microcontrôleur pilote désormais le jeu de manière autonome !

Malheureusement, pris par le temps à la toute fin du projet, nous n'avons pas eu l'occasion de générer le bitstream de cette dernière intégration pour la tester physiquement sur la carte ARTY. Cependant, comme le démontre sans ambiguïté le testbench ci-dessus, toute la logique et la synchronisation fonctionnent parfaitement.

Conclusion

En conclusion, ce projet difficile mais complet nous aura permis de réaliser le projet ambitieux de créer un jeu de zéro sur un FPGA. Nous avons fait face à la contrainte de travailler avec des signaux, apportant un nouveau niveau de complexité dans la programmation puisque nous devons tenir compte des cycles et front montant ou descendant lors de la conception de nos programmes.

Même s'il reste des points d'amélioration que nous avons évoqués plus haut, nous sommes très satisfaits d'avoir réussi à avoir un jeu fonctionnel qui implémente les deux parties, le microcontrôleur et le jeu.

Ce projet nous aura permis de bien comprendre la programmation VHDL car c'était un projet amusant à réaliser.